

DA3408 Assignment 2 Write-up

Vrishab A V, DA24B033

Q1

1) `Dockerfile.singlestage` installs all necessary build tools and dependencies, along with generating the dataset/model in one `python:3.11` layer, which itself is shipped as the runtime image. 2.15GB disk usage, 532MB content size.

2) In `Dockerfile.multistage` there is first a `builder` stage (full `python:3.11` which is just used for `pip install --target=/build/deps` and to train the model). After that the second `stage-1` image is built based on `python:3.11-slim`, which copies over only the installed dependencies, `model.joblib`, and `app.py`. The `multistage` removes the full `python:3.11` base image from the builder unlike the single-stage. Measured difference (terminal output in `evidence/q1_image_sizes.txt`):

Image	Disk usage	Content size
<code>spam-api-singlestage</code>	2.15GB	532MB
<code>spam-api-multistage</code>	729MB	166MB

Content size drops from 532MB to 166MB i.e. **68.8% reduction**, disk usage drops from 2.15GB to 729MB; 66.1% reduction.

3) The multi-stage image drops the full `python:3.11` base that was used by single-stage as its runtime and uses `python:3.11-slim` instead. Everything from the builder does not persist: `pip`'s wheel-build cache under `/root/.cache`, the build toolchain, and the generated `spam_dataset.csv` along with the training script. Only the installed `site-packages`, `model.joblib`, and `app.py` are there in the final image.

Additional Note:

`.dockerignore` (whose contents have been saved into `evidence/q1_dockerignore_context.txt`) excludes `__pycache__`, `*.pyc`, `outputs.md`, `spam_dataset.csv`, `model.joblib`, `evidence/`, `k8s/`, `.git`, and `*.md` from build context. The transferred context measured 3.05kB with and without it, since `Dockerfile.multistage` only `COPYs` four small files. The benefit of a `.dockerignore` is keeping those files out of the context as the repo grows, and doesn't save much on this particular run.

Q2

`/predict` hashes the input text with SHA-256 into a cache key and checks Redis first. On a MISS it runs the model, stores the label with a 300-second TTL (`CACHE_TTL_SECONDS`), and returns it; on a HIT it returns the stored label without recomputing. The JSON response (eg `{"label": "spam"}`) is the same whether hit or miss. Caching does not affect the client side; hit/miss is only visible server-side via `[cache] MISS/HIT` log lines.

`docker-compose.yml` declares `api` and `cache` (`redis:7-alpine`) on one network which is managed by Compose. Compose registers each service under its own name in DNS, so `api` reaches Redis at `cache:6379` with no additional setup required. 2 separate `docker run` containers would need an explicit user-defined network and linking to be done manually.

3) Measured effect (`evidence/q2_cache_benchmark.txt`), from `benchmark_cache.py` sending the same request twice:

Call	Result
Call 1 (cache MISS)	<code>{'label': 'spam'}</code> - 27.07 ms
Call 2 (cache HIT)	<code>{'label': 'spam'}</code> - 3.39 ms

The cached call (HIT) is ~8x faster since it skips model inferencing and returns straight from Redis.

4) Docker Compose's scope is narrower, as it runs multiple containers within a single Docker host, with one Docker daemon, one network and image cache, and no cluster, scheduler or additional machine. Kubernetes works on a larger scale to schedule and network containers across clusters of several machines; it has features like self-healing and rolling updates (as seen in Q3-Q4). We used Compose here since caching only needs 2 containers on one computer to cooperate; work is not spread across multiple minikube nodes on the cluster (which is where Kubernetes actually needs to come in).

Q3

`generate_shards.py` is the script I used to split synthetic signup data into 8 CSV shards with 50 rows each with roughly 20% of rows deliberately malformed (bad or missing email, missing date).

1) The cluster budget is 2 nodes $\times$ 2 CPUs = 4000m (4000 millicores = 4 cores). Each pod sets `requests` equal to `limits` at 500m CPU, so `parallelism: 4` uses $4 \times 500m = 2000m$ - half the cluster, 1000m out of each node's 2000m, leaving room for system pods. `parallelism: 8` would claim the entire 4000m with nothing spare, so pods would sit `Pending` behind anything else needing to be scheduled. 4 is therefore the largest value that still keeps each node at 50%.

`validate_shards.yaml` runs `validate_shard.py` with `completions: 8`, `parallelism: 4`, `completionMode: Indexed`, `restartPolicy: Never` and `backoffLimit: 2`. Kubernetes injects `JOB_COMPLETION_INDEX`, which the pod reads to pick `shards/shard_{idx}.csv`; `POD_NAME` and `NODE_NAME` come from the Downward API (`fieldRef: metadata.name`, `spec.nodeName`). A Job is the right object because this work finishes and exits rather than staying up.

2) `kubectl get pods -o wide` (`evidence/q3_job_pods.txt`) shows the run went in two waves of four (index 0-3 at age 3m26s, index 4-7 at 3m23s). This shows four pods really did run parallelly, spread across nodes `minikube-m02` and `minikube-m03`.

3) `collect_shard_results.py` asks the Kubernetes API for every pod belonging to this Job and reads each pod's log, then picks the result line out of it. I did not use a shared volume because minikube's default storage writes files into a folder on whichever node the pod happened to run on. A pod on one node and a pod on the other would each get their own separate folder, so something written by one pod would not be visible to the other. Reading logs goes through the API server instead, which works the same irrespective of which node the pod was on (`evidence/q3_shard_results.txt`).

Shard	Rows	Invalid	Node
0	50	11	minikube-m03
1	50	7	minikube-m02
2	50	11	minikube-m02
3	50	10	minikube-m03
4	50	8	minikube-m03
5	50	7	minikube-m02
6	50	7	minikube-m02
7	50	13	minikube-m03

400 rows were validated with 74 invalid across both worker nodes.

4) With 3 nodes $\times$ 2 CPUs = 6000m I would raise parallelism to 6: $6 \times 500m = 3000m$ keeps the same 50% leeway on a larger cluster. I would not go to 8 even though it would now fit, because there are only 8 shards in total; moving from 6 to 8 saves one short round at the very end, and in exchange every node is left with only a third of its CPU free instead of half.

Q4

1) `deployment.yaml` runs the spam API with 2 replicas. Each pod uses a quarter of a CPU and 256Mi of memory (capped at half a CPU, 512Mi). It probes `/healthz` for readiness every 5 seconds, starting 3 seconds after the pod comes up; until that probe passes, the Service sends no traffic to that pod. `service.yaml` is a NodePort service on port 30090 i.e every node in the cluster listens on that port and forwards to port 8000 inside whichever pods are ready.

2) I deleted one of the two running pods by name (demonstrated in the evidence files and in the video). Within a few seconds a new pod appeared with a different random suffix, and the other original pod was left completely alone. The ReplicaSet is what allows the Deployment to manage the pods like this; it counts how many pods carry the label `app=spam-api` and compares that number with the 2 it was supposed to keep. When the count dropped to 1 after 1 node was deleted it automatically created one more to make up that difference.

3) For the update I added `"version": "v2"` to the JSON output of `/healthz` returns, built that as a new image tag, and pointed the Deployment at it with `kubectrl set image`. `kubectrl rollout status` reported that the deployment successfully rolled out, and calling `/healthz` through the service changed from `{"status":"ok"}` to `{"status":"ok","version":"v2"}` (`evidence/q4_rollout.txt`). The update is gradual: the Deployment starts a new pod, waits for its readiness probe to pass, and only then removes an old one, so something is always answering.

One thing worth recording is that on a cluster with more than one node, building the image is not enough. My first attempt failed with `ErrImagePull` because the image existed on only one node. It has to be built with `docker build` and then copied to every node with `minikube image load`. `kubectrl rollout history` lists more than two revisions because the Deployment was updated several times - the failed attempt, and later a rollback to v1 so the update could be demonstrated again.

If this had been a Job instead of a Deployment there would be no rolling update at all. A Job has no ReplicaSet keeping two versions alive side by side, and no Service routing live traffic to it, so there is nothing to swap over gradually. Changing the code would mean deleting the Job and creating a new one, with a gap in between where nothing is running.

Because the requests here are lower than the limits, Kubernetes labels these pods **Burstable** (`evidence/q4_qos.txt`): they are guaranteed the amount they requested and may use more if the node has it spare. The Q3 job pods had their request and limit set to the same value, which makes them **Guaranteed** instead. A service handling uneven traffic benefits from being able to burst; a short fixed batch task does not.

`app.py` needs Redis, so `redis.yaml` adds a small Redis Deployment and Service named `cache` to the cluster. Without it `/predict` fails there while `/healthz` still passes, because `/healthz` never touches Redis. With it, `/predict` behaves exactly as it did under Compose - one MISS then one HIT (`evidence/q4_predict_check.txt`).

4) A Job fits work that finishes: each of the 8 shard pods does its piece and exits, and nothing should bring them back. A Deployment fits work that must not stop: the spam API has to keep answering requests, so it is worth having something watch the pod count, replace whatever dies, and swap versions over gradually instead of all at once. Using a Job for Q4 would mean a deleted pod stays dead and every update needs downtime; using a Deployment for Q3 would mean the shard pods get restarted forever after they had already finished their file. **limits**, these pods get **Burstable** QoS (`evidence/q4_qos.txt`), unlike the Q3 job's **Guaranteed** pods.

4) A Job is for work that ends: each of the 8 shard pods does its piece and exits, and nothing should restart them once they are done. A Deployment is for work that must not end - `spam-api` has to keep answering `/predict`, so it is worth having a controller replace failed pods and swap versions gradually. A Job in Q4 would mean no replacement and no rolling update; a Deployment in Q3 would leave 8 pods alive with nothing left to do.